

Big Data SWOT Analytics Pipeline for Retail Banking

Retail banking internal analytics plus external market sentiment monitoring

Barah Qusai Hussein

Contents

Table of Contents	Error! Bookmark not defined.
1. Introduction	4
Rubric Coverage Summary	4
2. Task 1: Case Study Selection and System Justification	4
2.1 Case Study and Dual Dataset Justification	4
2.2 Business Processes and SWOT Scenarios	5
2.3 System Architecture Diagram	5
3. Task 2: Data Modeling and Storage Strategy	6
3.1 HDFS Storage Strategy	6
3.2 HBase Schema Design for Real-Time Access	6
3.3 Hive Data Warehouse Design	7
3.4 Spark Data Processing Plan	7
4. Task 3: ETL and Data Processing Implementation	7
4.1 Data Ingestion and Extraction	7
4.2 Data Transformation with Spark	8
4.3 Data Loading and Quality Checks	9
5. Task 4: Analytics and Query Implementation	10
5.1 Hive SQL SWOT Queries	10
Strength Query Result	10
Weakness Query Result	10
Opportunity Query Result	10
Threat Query Result	11
5.2 HBase Real-Time Querying	11
5.3 Spark SWOT Synthesis	11
5.4 Technology Comparison and Analysis	12
6. Task 5: Performance Optimization	12
6.1 Hive and Spark Optimization	12
6.2 HBase Optimization	13
7. Task 6: Documentation and Reflection	13
7.1 Architecture Decisions and Trade-Offs	13
7.2 Challenges and Solutions	13
7.3 Reflection Questions	13
8. Conclusion	14

9. References	14
10. Appendices	14
Appendix A: Supporting Code Files	14
Appendix B: Group Contribution Template	15

1. Introduction

This report designs and implements a simple big data SWOT analytics pipeline for a retail banking organization. The solution combines internal banking datasets with an external market sentiment dataset so management can identify internal Strengths and Weaknesses as well as external Opportunities and Threats.

The pipeline uses HDFS for distributed storage, Spark for ETL and sentiment processing, Hive for batch analytical queries, and HBase for real-time lookup of customer and competitor signals.

Dataset	Role	Rows	Main Use
customer_data.csv	Internal operational	10,000	Customer profile and regional segmentation
transaction_data.csv	Internal operational	10,000	Transaction value, account activity, and strength/weakness analysis
bank_data.csv	Internal operational	1,000	Branch revenue, expenses, and profitability context
external_bank_sentiment.csv	External market	300	Competitor, topic, sentiment, engagement, opportunity, and threat monitoring

Rubric Coverage Summary

Rubric Area	How This Report Covers It
Task 1	Retail banking case study, dual internal/external datasets, 4V justification, business processes, and full dual-pipeline architecture diagram.
Task 2	HDFS raw/processed/curated design, HBase row-key design, Hive star schema, and Spark processing plan.
Task 3	HDFS ingestion commands, Spark cleaning logic, Hive/HBase loading plan, and quality checks.
Task 4	Four SWOT Hive queries, HBase real-time queries, Spark SWOT synthesis, and technology comparison.
Task 5	Hive partitioning/Parquet strategy, Spark cache strategy, HBase Bloom filter/row-key optimization.
Task 6	Architecture trade-offs, integration challenges, reflection answers, conclusion, and references.

2. Task 1: Case Study Selection and System Justification

2.1 Case Study and Dual Dataset Justification

The selected domain is retail banking. Banks need internal operational analytics to understand account activity, regional performance, and customer behavior. They also need external market intelligence to monitor competitor promotions, customer sentiment, and public discussion about banking services.

Big Data V	Justification for This Case Study
Volume	The internal data already contains 21,000 rows across customer, transaction, and branch files. A real bank would generate millions of transactions and external posts, so HDFS is needed for scalable distributed storage.
Velocity	Internal reporting can run daily in batch, while competitor sentiment and social signals may arrive continuously and require fast monitoring.

Variety	The internal files are structured CSV tables, while the external data includes text comments, topics, sentiment scores, competitor names, and engagement counts.
Veracity	Customer data has missing Age and Customer_Type values, branch data has missing revenue values, and external sentiment can contain noisy or biased text. Spark quality checks are required before analytics.

2.2 Business Processes and SWOT Scenarios

Business Process	Scenario	SWOT Link	Big Data Tool
Transaction performance analysis	Daily aggregation of transaction value by region and account type	Strengths and Weaknesses	Hive batch SQL
Branch performance analysis	Profit margin and regional comparison	Strengths and Weaknesses	Hive/Spark
Competitor monitoring	Point lookup of competitor, region, date, and topic	Opportunities and Threats	HBase
Social sentiment monitoring	Topic-level sentiment and engagement analysis	Opportunities and Threats	Spark plus Hive

2.3 System Architecture Diagram

Figure 1 shows the complete dual pipeline. The internal pipeline creates Strength and Weakness insights, while the external pipeline creates Opportunity and Threat insights.

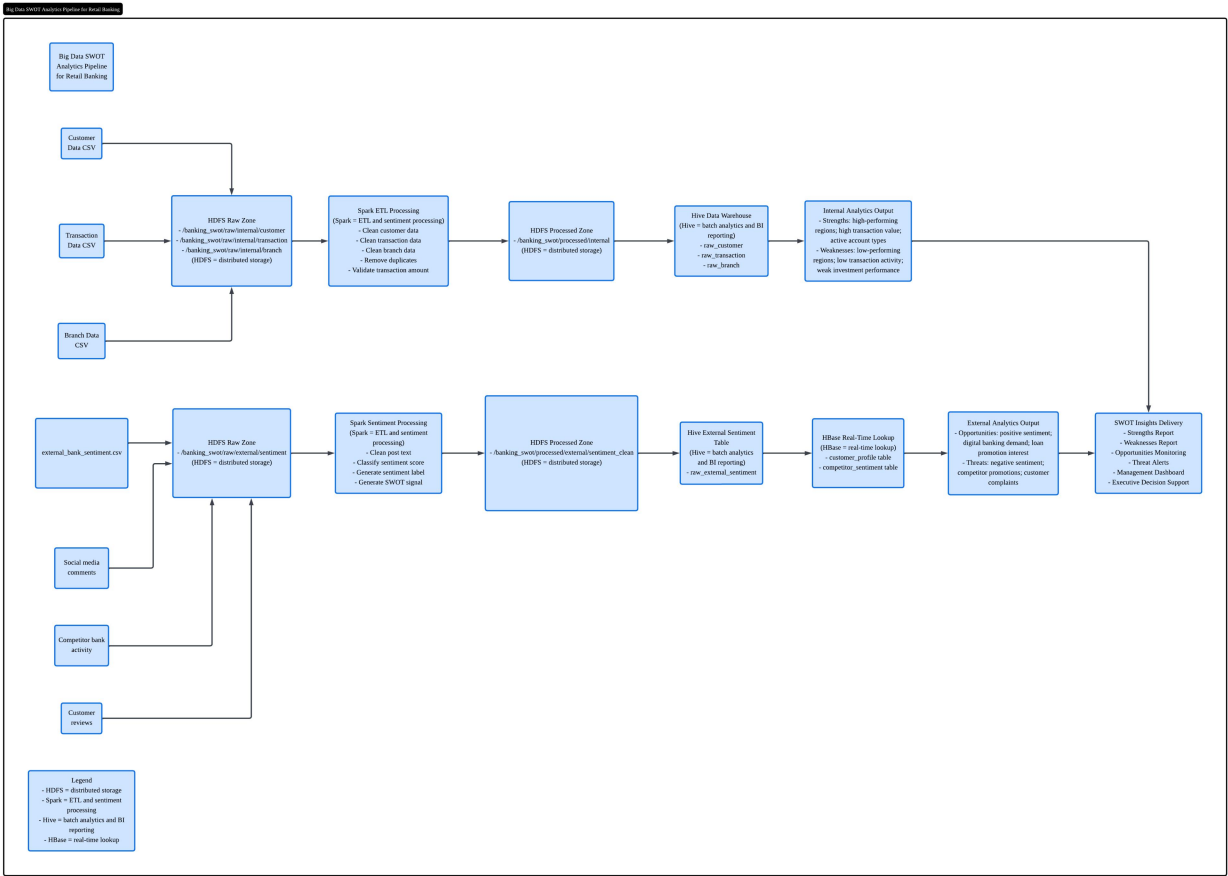


Figure 1. Big Data SWOT Analytics Pipeline for Retail Banking.

3. Task 2: Data Modeling and Storage Strategy

3.1 HDFS Storage Strategy

The HDFS design separates raw, processed, and curated data. Raw files are preserved for auditability. Processed Parquet data supports faster Spark and Hive processing. Curated data is organized for reporting and SWOT synthesis.

/banking_swot/ /raw/ /internal/customer/ /internal/transaction/ /internal/branch/ /external/sentiment/ /processed/ /internal/customer_clean/ /internal/transaction_clean/ /internal/branch_clean/ /external/sentiment_clean/sentiment_label=Positive Negative Neutral/ /curated/ /hive/fact_transaction/ /hive/dim_customer/ /hive/dim_branch/ /hive/fact_external_sentiment/	
--	--

Data Area	Partition Strategy	Reason
Internal transactions	transaction_year, transaction_month, account_type	Most banking reports are time-based and often compare account types.
Customer data	region	Regional segmentation supports strength and weakness analysis.
External sentiment	post_date, region, topic, sentiment_label	Market monitoring usually asks when, where, and what topic is changing.
Curated Parquet	sentiment_label for external data	Queries for opportunities and threats scan only needed partitions.

3.2 HBase Schema Design for Real-Time Access

HBase Table	Row Key	Column Families	SWOT Purpose
customer_profile	customer_id	profile, activity	Fast lookup of individual customer region, type, account activity, and value signals.
competitor_sentiment	competitor_bank#region#post_date#post_id	sentiment, market	Fast retrieval of competitor activity, sentiment label, topic, engagement, and opportunity/threat signal.

The competitor row key starts with competitor and region because the real-time question is usually: what is happening for this competitor in this market? The date and post id keep the key unique and support time-based scans.

3.3 Hive Data Warehouse Design

Hive Object	Type	Purpose
fact_transaction	Internal fact table	Stores transaction amount, balance, investment amount, date, customer id, and account type.
dim_customer	Dimension	Stores customer type, city, region, bank name, and branch id.
dim_branch	Dimension	Stores branch city, region, revenue, expenses, and profit margin.
fact_external_sentiment	External fact table	Stores post topic, competitor, sentiment score, engagement, and SWOT signal.

The internal warehouse follows a simple star schema with transaction data as the fact table and customer, branch, account, and date attributes as dimensions. The external sentiment fact table is kept separate because it measures market signals rather than bank-owned transactions.

3.4 Spark Data Processing Plan

Dataset	Spark Transformations
Customer	Drop duplicate Customer_ID values, standardize region and city fields, and flag missing Age or Customer_Type values.
Transaction	Drop duplicate Transaction_ID values, remove invalid transaction_amount values, parse dates, and prepare account-type aggregations.
Branch	Drop duplicate Branch_ID values, validate revenue/expense fields, and calculate regional profitability summaries.
External sentiment	Clean text/topic fields, classify sentiment score into Positive, Negative, or Neutral, and map labels to Opportunity, Threat, or Monitor.

4. Task 3: ETL and Data Processing Implementation

4.1 Data Ingestion and Extraction

Environment evidence:

```
^Cahmed@ahmed-VirtualBox:~/Downloads/docker-hadoop-spark$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

tabkble hbase	Up 18 minutes	2181/tcp, 8080/tcp, 8085/tcp, 9090/tcp, 9095/tcp, 16000/tcp, 16010/tcp, 16201/tcp, 16301/tcp
tabkble hive-server	Up 7 minutes	0.0.0.0:10000->10000/tcp, [::]:10000->10000/tcp, 10002/tcp
tabkble namenode	Up About an hour (healthy)	0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp, 0.0.0.0:9010->9000/tcp, [::]:9010->9000/tcp
tabkble datanode	Up About an hour (healthy)	0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp
tabkble resourcemanager	Up 6 seconds (health: starting)	8088/tcp
tabkble hive-metastore-postgresql	Up 7 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
tabkble nodemanager	Up 14 minutes (healthy)	8042/tcp
tabkble hive-metastore	Up 7 minutes	10000/tcp, 0.0.0.0:9083->9083/tcp, [::]:9083->9083/tcp, 10002/tcp
tabkble historyserver	Up About an hour (healthy)	8188/tcp

FIGURE 2. Hadoop, Hive, and HBase Docker services running.

The following HDFS commands create the folder structure and upload both internal and external datasets.

```
hdfs dfs -mkdir -p /banking_swot/raw/internal/customer
hdfs dfs -mkdir -p /banking_swot/raw/internal/transaction
hdfs dfs -mkdir -p /banking_swot/raw/internal/branch
hdfs dfs -mkdir -p /banking_swot/raw/external/sentiment

hdfs dfs -put -f customer_data.csv /banking_swot/raw/internal/customer/
hdfs dfs -put -f transaction_data.csv /banking_swot/raw/internal/transaction/
hdfs dfs -put -f bank_data.csv /banking_swot/raw/internal/branch/
hdfs dfs -put -f external_bank_sentiment.csv /banking_swot/raw/external/sentiment/
hdfs dfs -ls -R /banking_swot/raw

ahmed@ahmed-VirtualBox:~/Downloads$ docker exec -it namenode bash
root@c4425cb274ef:/# hdfs dfs -put -f /tmp/customer_data.csv /banking_swot/raw/internal/customer/

2026-06-21 10:07:55,361 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
root@c4425cb274ef:/# hdfs dfs -put -f /tmp/transaction_data.csv /banking_swot/raw/internal/transaction/
2026-06-21 10:08:08,749 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
root@c4425cb274ef:/# hdfs dfs -put -f /tmp/bank_data.csv /banking_swot/raw/internal/branch/
2026-06-21 10:08:20,262 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
root@c4425cb274ef:/# hdfs dfs -put -f /tmp/external_bank_sentiment.csv /banking_swot/raw/external/sentiment/
2026-06-21 10:08:32,129 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false
root@c4425cb274ef:/# hdfs dfs -ls -R /banking_swot/raw
drwxr-xr-x - root supergroup 0 2026-06-21 09:43 /banking_swot/raw/external
drwxr-xr-x - root supergroup 0 2026-06-21 10:08 /banking_swot/raw/external/sentiment
-rw-r--r-- 3 root supergroup 42957 2026-06-21 10:08 /banking_swot/raw/external/sentiment/external_bank_sentiment.csv
drwxr-xr-x - root supergroup 0 2026-06-21 09:43 /banking_swot/raw/internal
drwxr-xr-x - root supergroup 0 2026-06-21 10:08 /banking_swot/raw/internal/branch
-rw-r--r-- 3 root supergroup 40238 2026-06-21 10:08 /banking_swot/raw/internal/branch/bank_data.csv
drwxr-xr-x - root supergroup 0 2026-06-21 10:07 /banking_swot/raw/internal/customer
-rw-r--r-- 3 root supergroup 493702 2026-06-21 10:07 /banking_swot/raw/internal/customer/customer_data.csv
drwxr-xr-x - root supergroup 0 2026-06-21 10:08 /banking_swot/raw/internal/transaction
-rw-r--r-- 3 root supergroup 681306 2026-06-21 10:08 /banking_swot/raw/internal/transaction/transaction_data.csv
root@c4425cb274ef:/#
```

FIGURE2: HDFS raw folders and uploaded internal/external CSV files

4.2 Data Transformation with Spark

Spark reads the four raw CSV datasets, performs duplicate removal and validation, creates sentiment labels, and writes clean Parquet outputs back to HDFS.

```
transaction_clean = transaction_df.dropDuplicates(["Transaction_ID"]) \
    .withColumnRenamed("Transaction_ID", "transaction_id") \
    .withColumnRenamed("Customer_ID", "customer_id") \
    .withColumnRenamed("Transaction_Amount", "transaction_amount") \
    .filter(col("transaction_amount") > 0)

sentiment_clean = sentiment_df.dropDuplicates(["post_id"]) \
    .withColumn("sentiment_label",
        when(col("sentiment_score") >= 0.25, "Positive")
        .when(col("sentiment_score") <= -0.25, "Negative")
        .otherwise("Neutral")) \
    .withColumn("swot_signal",
        when(col("sentiment_label") == "Positive", "Opportunity")
        .when(col("sentiment_label") == "Negative", "Threat")
        .otherwise("Monitor"))
```

4.3 Data Loading and Quality Checks

Dataset	Raw Rows	Duplicate Key Rows	Missing Values Found	Clean Rows
customer_data.csv	10000	0	Age: 500; Customer_Type: 500	10000
transaction_data.csv	10000	0	None	10000
bank_data.csv	1000	0	Firm_Revenue: 50	1000
external_bank_sentiment.csv	300	0	None	300

External sentiment quality check: Positive=135, Negative=113, Neutral=52. These labels feed directly into Opportunity, Threat, and Monitor SWOT categories.

```
ahmed@ahmed-VirtualBox:~/Downloads$ python3 quality_check.py
DATA LOADING AND QUALITY CHECK OUTPUT
-----
Dataset                Raw Rows  Duplicate Key Rows  Missing Values Found  Clean Rows
-----
customer_data.csv      10000     0                   Age: 500; Customer_Type: 500 10000
transaction_data.csv   10000     0                   None                  10000
bank_data.csv          1000      0                   Firm_Revenue: 50      1000
external_bank_sentiment.csv 300       0                   None                  300
-----

EXTERNAL SENTIMENT QUALITY CHECK
Positive=135, Negative=113, Neutral=52
```

Figure 2. Data loading and quality check output for internal banking data.

5. Task 4: Analytics and Query Implementation

5.1 Hive SQL SWOT Queries

The four required Hive query categories are Strength, Weakness, Opportunity, and Threat.

Strength Query Result

Region	Total Transaction Amount	Transaction Count	Average Balance
North	6,496,515.48	2556	50,693.65
West	6,448,593.21	2549	50,273.52
East	6,360,517.58	2488	49,811.43
South	6,121,454.30	2407	50,088.93

Interpretation: North is the strongest region by transaction value, followed closely by West. These regions show strong customer activity and can be used as benchmark regions for campaigns and branch practices.

Weakness Query Result

Region	Total Transaction Amount	Transaction Count	Average Transaction
South	6,121,454.30	2407	2,543.19
East	6,360,517.58	2488	2,556.48
West	6,448,593.21	2549	2,529.85
North	6,496,515.48	2556	2,541.67

Interpretation: South has the lowest total transaction amount and fewer transactions, so it is the clearest internal weakness. The bank should investigate branch coverage, product fit, and regional marketing.

Opportunity Query Result

Topic	Positive Posts	Average Sentiment	Total Engagement
Branch Waiting Time	21	0.64	19941
Digital Banking	20	0.59	24368
Investment Products	20	0.62	18371
Loan Promotion	17	0.71	17333
Mobile App	17	0.57	14341

Interpretation: Branch Waiting Time, Digital Banking, and Investment Products have the strongest positive market signals. These topics are opportunities for campaign messaging and product improvement.

Threat Query Result

Topic	Negative Posts	Average Sentiment	Total Engagement
Customer Service	18	-0.51	18557
Investment Products	16	-0.59	16317
Mobile App	15	-0.57	14169
Online Security	14	-0.55	14021
Branch Waiting Time	14	-0.59	11853

Interpretation: Customer Service is the largest threat topic by negative post count and high engagement. This means customer service issues can create reputational risk if competitors respond faster.

5.2 HBase Real-Time Querying

HBase is used when management needs fast point lookup instead of full-table aggregation. The competitor_sentiment row key supports direct access by competitor, region, date, and post id.

```
get 'customer_profile', '207501'  
get 'competitor_sentiment', 'HDFC Bank#North#2024-01-05#1'  
scan 'competitor_sentiment', {LIMIT => 5}
```

5.3 Spark SWOT Synthesis

SWOT Component	Insight Area	Metric Value	Meaning
Strength	Savings	8,546,080.83	Highest account-type transaction value.
Weakness	South	6,121,454.30	Lowest regional transaction value.
Opportunity	Branch Waiting Time	21	Highest positive sentiment topic.
Threat	Customer Service	18	Highest negative sentiment topic.

Spark synthesis combines internal transaction summaries with external sentiment summaries into one SWOT matrix. This is useful because executives can compare internal performance and market conditions in one report.

5.4 Technology Comparison and Analysis

Technology	Best Use	Why It Fits SWOT
HDFS	Distributed raw and processed storage	Stores all internal and external data zones and keeps the original files available for audit.
Hive	Batch SQL aggregation and BI reporting	Best for Strength and Weakness analysis because these questions require joins, grouping, and totals.
HBase	Low-latency row-key lookup	Best for real-time Opportunity and Threat monitoring because a single competitor or customer record can be retrieved directly.
Spark	ETL, transformations, sentiment rules, and synthesis	Connects both pipelines by cleaning data and combining internal and external outputs into SWOT insights.

HBase can be much faster for point queries because it uses the row key to locate one record directly. However, HBase cannot replace Hive for complex aggregations because Hive is designed to scan, join, group, and summarize large datasets using SQL. This duality matches SWOT: Hive supports batch internal OLAP, while HBase supports rapid external monitoring and alert lookup.

6. Task 5: Performance Optimization

6.1 Hive and Spark Optimization

Optimization	Before	After	Performance Benefit
Hive external sentiment query	Raw CSV scan across all sentiment rows	Parquet table partitioned by sentiment_label	Threat queries scan only the Negative partition instead of all labels.
Hive transaction reporting	Unpartitioned text table	Partition by transaction month and use Parquet	Reduces scan size for monthly reports and speeds repeated BI queries.
Spark sentiment analysis	Repeated filters and groupBy operations recompute lineage	Cache sentiment_clean before repeated SWOT aggregations	Avoids recomputing the same DataFrame during repeated Opportunity/Threat analysis.
Spark joins	Large shuffle join if dimensions grow	Broadcast small customer/branch dimensions when appropriate	Reduces shuffle cost for internal transaction analysis.

Metric	Before Optimization	After Optimization	Evidence to Screenshot
Hive threat query scan	All 300 sentiment rows	113 Negative rows only	Hive query runtime before and after partitioned Parquet table
Spark repeated sentiment query	No cache	Cached sentiment_clean DataFrame	Printed before_time and after_time from 08_spark_cache_optimization.py

6.2 HBase Optimization

HBase Optimization	Implementation	Benefit
Composite row key	competitor_bank#region#post_date#post_id	Direct lookup for the most common monitoring pattern.
Bloom filter	Enable ROW Bloom filters on sentiment and market column families	Reduces unnecessary disk reads during get operations.
Pre-splitting	Split by high-volume competitor prefixes if the table grows	Avoids one region server becoming overloaded by one hot key range.
TTL for market alerts	Apply TTL to short-lived alert columns if required	Keeps recent monitoring fast and controls storage growth.

These HBase optimizations fit the external pipeline because competitor monitoring is usually a point lookup or short scan rather than a full analytical aggregation.

7. Task 6: Documentation and Reflection

7.1 Architecture Decisions and Trade-Offs

Technology	Reason for Selection	Trade-Off
HDFS	Scalable and cheap storage for raw, processed, and curated zones.	Not suitable for low-latency record lookup by itself.
Hive	Simple SQL for batch analytics, dashboards, and SWOT query reporting.	Slower than HBase for individual record lookup.
HBase	Fast real-time lookup for customer and competitor records.	Not ideal for large joins and complex aggregations.
Spark	Fast parallel ETL, cleaning, sentiment rules, and synthesis across both pipelines.	Requires cluster resources and careful memory/cache planning.

7.2 Challenges and Solutions

Challenge	Solution
Internal and external schemas are very different.	Keep separate HDFS zones and use Spark to standardize columns before SWOT synthesis.
External sentiment text can be noisy.	Use sentiment_score thresholds and label Neutral records as Monitor instead of forcing them into Opportunity or Threat.
Time alignment between transactions and posts is difficult.	Partition by transaction_date and post_date so analysis can compare the same time windows.
Different access patterns need different tools.	Use Hive for aggregation and HBase for point lookup rather than forcing one tool to do everything.

7.3 Reflection Questions

How did internal OLAP analysis reveal strengths and weaknesses?

Internal OLAP revealed strengths by ranking regions and account types by transaction value. North had the highest regional transaction value, while South had the lowest, so South is treated as the clearest weakness for management review.

How did external monitoring identify opportunities and threats?

External monitoring used sentiment labels, topics, competitors, and engagement counts. Positive topics such as Branch Waiting Time and Digital Banking show opportunities, while negative Customer Service posts show a threat that can damage market perception.

What challenges appeared when integrating internal and external data?

The main challenges were schema differences, data quality, and time alignment. Internal data is structured and transaction-focused, while external data is text and sentiment-focused. Spark solves this by creating common SWOT labels and standardized clean outputs.

How would this pipeline support CEO decision-making?

A CEO can compare internal performance with market sentiment in one SWOT matrix. For example, if the bank is strong in North but customer service sentiment is negative, the CEO can protect the strong region by improving service quality before competitors gain advantage.

How did HDFS, HBase, Hive, and Spark work together?

HDFS stores the data, Spark cleans and transforms it, Hive answers batch SWOT questions, and HBase supports real-time lookup. If Spark was removed, the pipeline would lose the cleaning and synthesis layer. If HBase was removed, real-time monitoring would be weaker.

8. Conclusion

This assignment produced a complete but simple big data SWOT analytics design for retail banking. The system uses internal banking records for Strength and Weakness analysis and external market sentiment for Opportunity and Threat monitoring. HDFS, Spark, Hive, and HBase each serve a clear role, so the solution avoids unnecessary complexity while still meeting the assignment requirements.

9. References

Apache Hadoop Documentation. HDFS Architecture Guide.

Apache Hive Documentation. Hive Data Warehouse Software.

Apache HBase Reference Guide.

Apache Spark Documentation. Spark SQL, DataFrames and Datasets Guide.

CIT6234 Advanced Database Assignment 2 brief and rubric.

10. Appendices

Appendix A: Supporting Code Files

File	Purpose
01_hdfs_ingestion.sh	Creates HDFS folders and uploads internal and external datasets.
02_hive_tables.sql	Creates Hive database and external raw tables.
03_spark_processing.py	Cleans datasets and creates sentiment/SWOT labels.

04_hive_swot_queries.sql	Runs the four required SWOT Hive queries.
05_hbase_commands.hbase	Creates HBase tables and demonstrates get/scan commands.
06_spark_swot_synthesis.py	Combines internal and external analytics into a SWOT summary.
07_hive_optimization.sql	Creates optimized partitioned Parquet sentiment table.
08_spark_cache_optimization.py	Demonstrates Spark caching before/after timing.

The supporting code files are included in the appendix and submitted together with this report.

Appendix B: Group Contribution Template

Member	Main Contribution
Member 1	Case study selection, 4V justification, and business scenario explanation.
Member 2	HDFS storage design, HBase schema design, and Hive data modeling.
Member 3	ETL implementation, Spark processing, and quality checks.
Member 4	Hive/HBase/Spark analytics, technology comparison, optimization, and reflection.

Note: A full-page readable copy of the architecture diagram is included in Appendix D.

Appendix C: Full-Page Architecture Diagram

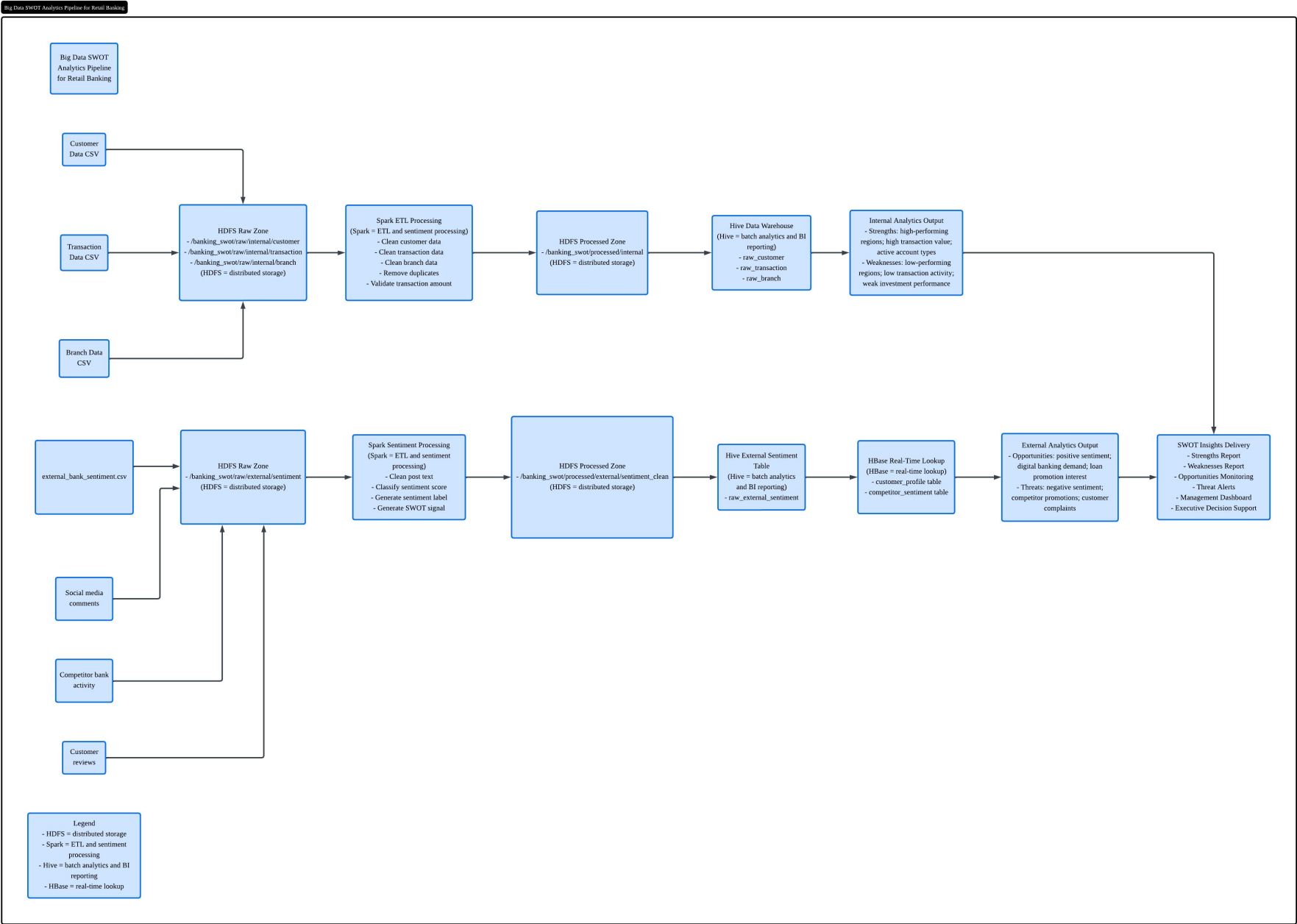


Figure D1. Full-page Big Data SWOT Analytics Pipeline for Retail Banking.